

Intelligence Layer

Concept Design Document

Domain-Aware, Model-Routed, Multi-Agent Reasoning for the Regulated Enterprise
Phase 2 Concept | Confidential | 2026

Fortune 500 and Regulated Enterprises Target Market	SaaS Subscription and Usage Model	Phase 2 Build Stage
---	---	-------------------------------

Executive Summary

Aligns with: Architecture, Layer 3 Intelligence Layer

Enterprises that solve isolation, identity, and orchestration still fail in production because the reasoning layer is ad hoc: one-off prompts, ungrounded LLM calls, no regression discipline, and domain logic trapped in notebooks.

In healthcare, P&C, and payment integrity specifically, the result is pilots that cannot survive audit, throughput, or precision bars.

What the Intelligence Layer Delivers

- Domain-aware, model-routed, multi-agent reasoning plane built on secure and orchestration infrastructure
- Measurable outcomes: accuracy, latency, and cost per decision
- Evidence suitable for regulated sign-off across healthcare, payment integrity, and financial services
- Productizable across agent platform partners and cost containment suites

One line: Orchestration moves work. Intelligence decides work correctly, explainably, and at scale.

The Problem

Aligns with: Market Problem, Production Failure Modes

Secure and Orchestration layers are built. The Intelligence Layer closes the final gap: ungrounded reasoning, no hybrid routing, single-agent fragility, and the absence of evaluation discipline that prevents pilots from reaching production.

Failure Modes the Intelligence Layer Resolves

- Ungrounded generation on claims and clinical text, which leads to false adjudication and compliance exposure
- No hybrid routing (LLM for everything), causing cost blowups in high-throughput bill streams
- Single-agent prompt chains: fragile long workflows with no role separation
- Weak eval (manual spot checks) cannot promote POC to enterprise; no regression on policy change
- Retrieval treated as optional, resulting in hallucinated codes, fees, or clinical rationales
- Model and version drift invisible, causing silent quality decay; audit cannot reconstruct decision rationale

Regulated enterprise deployments require not just agent capability but measurable, auditable, explainable decision quality, a bar that ad hoc prompt engineering cannot meet.

Product Vision

Aligns with: Product Overview, Reusable Reasoning Substrate

The Intelligence Layer is a reusable reasoning substrate, not a replacement for secure isolation or orchestration, not a chat UI or copilot wrapper, and not RAG bolted on without eval or schema-bound outputs.

Core Capabilities

- **ROUTE:** Routes each work unit to the right mix of rules, retrieval, and models. Deterministic-first.
- **COORDINATE:** Specialist agents (retrieve, classify, reason, structure, escalate) inside orchestration boundaries.
- **GROUND:** Outputs grounded in tenant-scoped corpora: policies, fee schedules, clinical guidelines, historical decisions.
- **SCORE:** Every material output scored for grounding, policy adherence, and regression baselines.
- **EMIT:** Structured artifacts and explainability payloads written to the secure audit stream.

Primary Buyers and Wedges

Segment	Wedge Workflows	Success Metrics
Agent Platform / SI	Ticket triage, incident enrichment, doc-heavy operations	Time-to-resolution, escalation rate, audit pass rate
Payment Integrity	Line-level adjudication, FWA triage, clinical and pharmacy review	Precision and recall on flags, savings recovered, analyst time per claim
Healthcare Adjacent	Prior auth support, coding suggestion with human-in-loop	Override rate, compliance findings, latency p95

Architecture and Trust Boundaries

Aligns with: Architecture, Shared Control Plane and Per-Tenant Runtime

The Intelligence Layer operates as a third discrete service domain above the Secure and Orchestration layers. A shared control plane manages identity, policy, scheduling, and telemetry. Per-tenant runtimes execute agent workloads in isolated environments with no shared memory and no cross-tenant data paths.

Three-Layer Stack

LAYER 3: INTELLIGENCE (BUILD NEXT) Policy-aware hybrid router · Specialist agent mesh · Domain RAG · Eval plane · Model health · Structured outcomes and explainability
LAYER 2: ORCHESTRATION (BUILT) Schedule · State machines · Checkpoints · Recovery · Tool contracts · Metering
LAYER 1: SECURE (BUILT) Tenant isolation · Identity before execution · Policy gates · PHI/PII boundaries · Short-lived credentials · Audit event retention

AI Agent Architecture Overview

The diagram below shows the full agent execution topology: input channels entering the Orchestrator, routing to the AI Agent layer, source control triggers from GitHub, and output scrubbing through the DMZ before delivery.

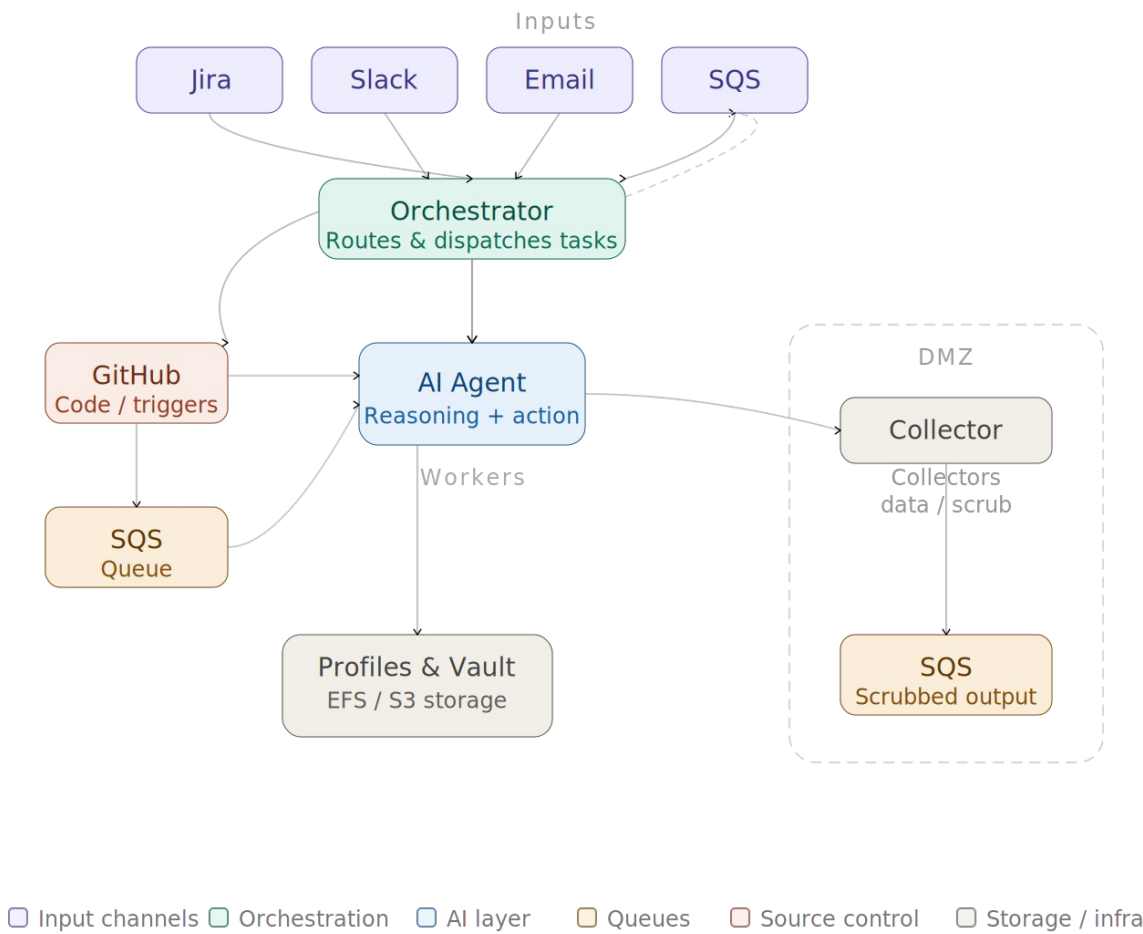


Figure 1: AI Agent Architecture — Input Channels, Orchestration, Agent Execution, and DMZ Output

Intelligence Layer Components

Component	Function	Design Principle
Hybrid router	Classify work unit into path: rule-only, retrieve-then-structure, multi-agent graph, or human escalation	Deterministic when confidence high; LLM only inside template bounds
Agent mesh	Role-specific agents: Retriever, Classifier, Reasoner, Formatter, Escalation	LangGraph or custom FSM; no unbounded agent-to-agent chatter
Domain RAG	Embeddings and metadata filters over unstructured medical, claims, and policy text	Tenant-scoped indexes; retrieval tuned per vertical corpus
Tool intelligence	MCP/API/DB calls with structured outputs including JSON Schema and claim line objects	Fail closed; tool allowlist enforced from secure layer

Component	Function	Design Principle
Guardrails	Injection defense, PHI leakage checks, output policy validators	Runtime enforcement, not post-hoc only
Eval plane	Hallucination and grounding checks, golden-set regression, model health scoring	Gates POC-to-production promotion; versioned with policy packs
Model registry	Primary model, fallbacks, pinning, cost and latency budgets	Per-tenant routing rules; health dashboards for governance review
Long-context strategy	Summarization, external memory, and checkpoint refs	Orchestration holds thread state; avoids unbounded context stuffing

API Contract: Downstream of Orchestration

Orchestration passes an approved run spec to the Intelligence Layer. The layer returns a structured decision artifact with explainability and eval scores attached.

Run Spec (Orchestration to Intelligence)

- tenant_id, run_id, workflow_template_id, policy_version
- data_scope: which indexes and corpora are in scope for this run
- tool_allowlist: permitted tool calls for this execution
- output_schema_id: target structured output schema
- sla_tier: interactive or batch
- pii_mode: strict enforcement flag

Decision Artifact (Intelligence to Orchestration)

- status: completed, escalated, or failed
- structured_output: schema-conformant result object
- explainability: citations, rule hits, agent trace summary
- eval_scores: grounding score and policy adherence score
- cost: token count and model ID used

Core Capabilities

Aligns with: Product Capabilities, Routing, Multi-Agent, Grounding, and Eval

Intelligent Routing

The hybrid router is the product centerpiece: deterministic-first, with LLM invoked only where needed. It classifies each work unit using a feature vector (document type, line count, clinical specialty, dollar band, and policy version) to select the optimal execution path. Financial and clinical high-risk paths default to retrieve-first with a human-in-loop threshold before any structured output is written.

Multi-Agent Workflow Patterns

Pattern	Agents	Primary Use
Retrieve, Reason, Structure	Retriever, Analyzer, Formatter	Adjudication assist, clinical review
Classify and Branch	Classifier plus Routers	FWA triage, routing to specialty queues
Extract, Validate, Score	Extractor, Validator, Scorer	Unstructured bill fields, edits check assist
Synthesize and Audit	Synthesizer, Quality Gate	Executive summaries with eval gate

Grounding and Data

Ingestion adapters support structured claims and unstructured content including EOBs, clinical notes, contracts, and PDF documents. All outputs are normalized to tenant canonical schemas. Mandatory citation is required on any code, dollar figure, or clinical assertion appearing in the output schema; no assertion is permitted without a retrievable source.

Evaluation and Model Health

Eval Types
- Golden-set regression: block deploy on policy or model change if metrics drop below baseline - Grounding and hallucination: citation coverage, entailment checks on retrieved spans - Precision at k (domain): flag precision on labeled fraud, waste, and abuse samples - Latency and cost: p95 against SLA tier; cost per 1K lines processed - Model health score: composite drift signal for governance review

Fine-Tuning Trajectory

Phase	Scope
Now	Embedding fine-tune (triplet loss) for retrieval quality; prompt and schema discipline

Phase	Scope
Next	LoRA and SFT on narrow tasks (line coding, document classification) inside tenant boundary
Later	RLHF and GRPO only where human feedback loop and full audit trail are in place

MCP Integration: Agent Identity and Tool Workflow

Aligns with: Model Context Protocol, Agent Identity, Tool Bus, and Platform Workflow

The Model Context Protocol (MCP) is the tool contract standard that governs every external call an agent makes. In this platform, MCP is not just a developer convenience. It is the identity and policy enforcement point for every tool invocation across the entire agent mesh.

Why MCP Matters for Regulated Platforms

- Every tool call carries a verified agent identity token; no anonymous or ambient tool access is permitted
- The allowlist in the run spec is enforced per call; agents cannot invoke tools not approved for that run
- Every call produces a structured audit event: agent ID, tool called, input hash, output hash, eval score, and timestamp
- Schema validation is applied to every tool response before it reaches the agent; malformed or out-of-spec responses are rejected
- Fail-closed enforcement means any violation (bad auth, wrong tool, schema mismatch) triggers escalation, never a silent pass

MCP Architecture: Agent Mesh, Dispatcher, and Server Catalog

The diagram below shows the full MCP topology: agents in the mesh each carry an identity token into every call; the MCP Dispatcher enforces identity, allowlist, and schema before routing to the appropriate server; the audit stream and fail-closed layer sit below every invocation.

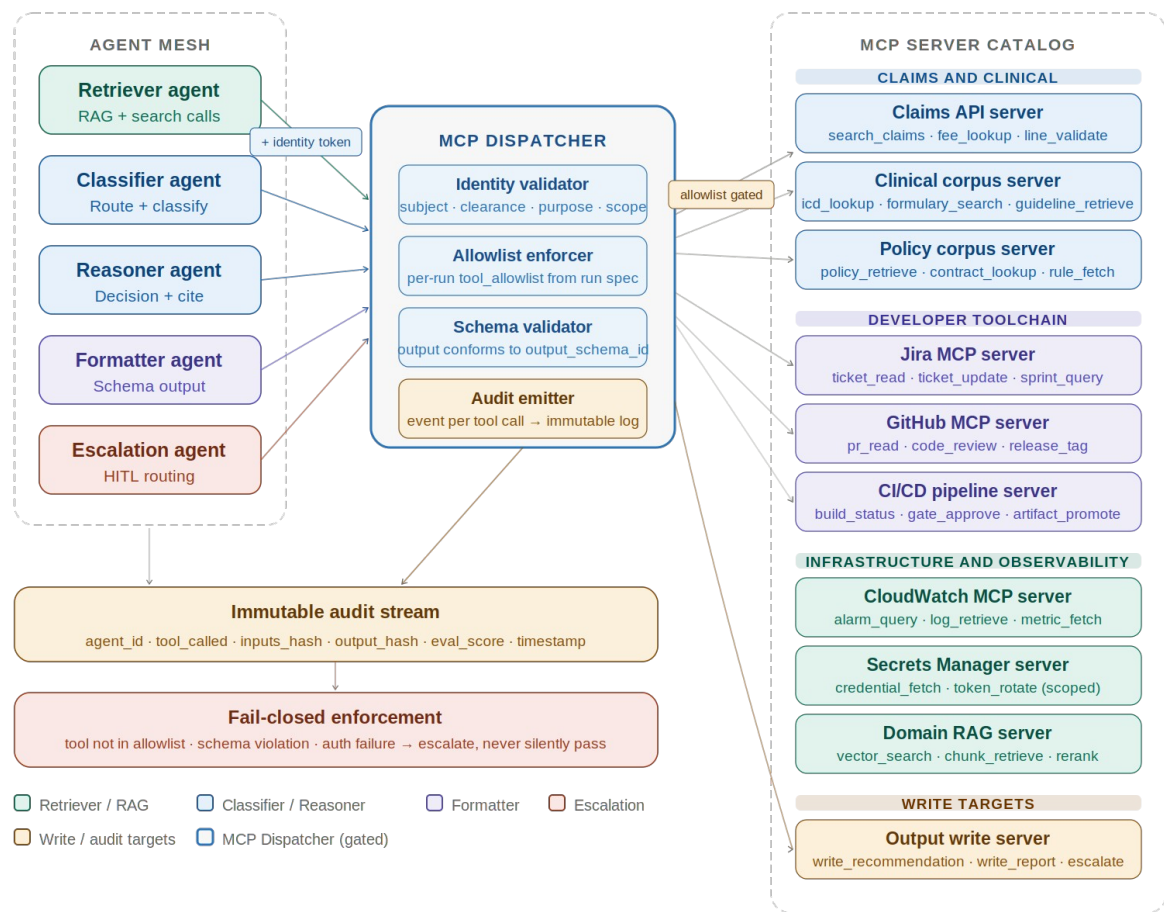


Figure 4: MCP Architecture — Agent Identity, Dispatcher Gates, Server Catalog, and Audit Stream

Agent Identity Model

Every agent in the mesh is assigned a structured identity at the start of each run. This identity is not inherited from the human user session. It is issued by the Orchestrator as a scoped, time-limited token tied to the specific run spec.

Identity Field	Source	Enforced At
subject (agent_id)	Issued by Orchestrator at run start; unique per run	MCP Dispatcher identity validator on every tool call
clearance (data tier)	Derived from run spec pii_mode and data_scope	MCP Dispatcher before routing to any restricted server
purpose (task code)	Inherited from workflow_template_id in run spec	MCP Dispatcher purpose check; tool server purpose allowlist
scope (tool allowlist)	Explicit tool_allowlist field in run spec	MCP Dispatcher allowlist enforcer; rejects unlisted calls
ttl (expiry)	Set at run start; expires at run completion or timeout	Orchestrator credential manager; token rejected after expiry

Identity Field	Source	Enforced At
run_id (correlation)	Carried on every call for full trace correlation	Immutable audit log; links every tool call to originating run

MCP Server Catalog

The server catalog defines the available tool servers, their exposed methods, and the identity clearance level required to invoke them. Servers are registered at platform configuration time; no agent can call a server that is not in the catalog.

Server	Category	Exposed Methods	Required Clearance
Claims API server	Claims and clinical	search_claims, fee_lookup, line_validate	Restricted (HIPAA)
Clinical corpus server	Claims and clinical	icd_lookup, formulary_search, guideline_retrieve	Restricted (HIPAA)
Policy corpus server	Claims and clinical	policy_retrieve, contract_lookup, rule_fetch	Standard
Jira MCP server	Developer toolchain	ticket_read, ticket_update, sprint_query	Standard
GitHub MCP server	Developer toolchain	pr_read, code_review, release_tag	Standard
CI/CD pipeline server	Developer toolchain	build_status, gate_approve, artifact_promote	Elevated (pipeline role)
CloudWatch MCP server	Infrastructure	alarm_query, log_retrieve, metric_fetch	Standard
Secrets Manager server	Infrastructure	credential_fetch, token_rotate (scoped only)	Elevated (ops role)
Domain RAG server	Retrieval	vector_search, chunk_retrieve, rerank	Derived from data_scope
Output write server	Write targets	write_recommendation, write_report, escalate	Standard (schema-gated)

MCP Tool Call Lifecycle

The following sequence describes the complete lifecycle of a single MCP tool call, from the agent decision through policy enforcement, server execution, schema validation, and audit emission.

Step-by-Step: MCP Tool Call Lifecycle

- Step 1: Agent decision. The Reasoner or Retriever agent determines a tool call is needed and constructs a call request with method name, input parameters, and its identity token.
- Step 2: Dispatcher identity check. The MCP Dispatcher validates the identity token (subject, clearance, purpose, ttl) against the run spec; expired or invalid tokens are rejected with a 401

audit event.

- Step 3: Allowlist enforcement. The Dispatcher checks the requested method against the `tool_allowlist` in the run spec; any method not explicitly listed is rejected with a 403 event and an audit entry.
- Step 4: Server routing. The Dispatcher routes the validated call to the appropriate MCP server; the server receives the agent identity alongside the call parameters.
- Step 5: Server execution. The tool server executes the method (database query, API call, vector search, or file write) and returns a raw result.
- Step 6: Schema validation. The Dispatcher validates the raw result against the `output_schema_id` from the run spec; schema violations trigger rejection and escalation.
- Step 7: Audit emission. The Dispatcher writes an immutable audit event containing `agent_id`, `tool_called`, `inputs_hash`, `output_hash`, `eval_score`, and `timestamp` to the log archive account.
- Step 8: Result delivery. The validated, schema-conformant result is returned to the requesting agent for inclusion in its reasoning step.

Agent Plugins: Defining and Registering a Tool Workflow

An agent plugin is a versioned, schema-bound definition of a tool workflow that a specific agent type is permitted to execute. Plugins are registered with the platform at deploy time and referenced by workflow templates. Every plugin must declare its method set, input schema, output schema, required clearance level, and fail behavior.

Plugin Definition Fields

- `plugin_id`: unique versioned identifier, for example `claims_adjudication_retriever_v2`
- `agent_type`: the agent class this plugin is available to (retriever, reasoner, formatter, or escalation)
- `method_set`: the exact list of MCP methods this plugin may invoke, which maps to `tool_allowlist` in the run spec
- `input_schema`: JSON Schema defining acceptable input parameters for each method
- `output_schema`: JSON Schema the tool response must conform to before being returned to the agent
- `required_clearance`: minimum clearance level the agent identity must carry to use this plugin
- `fail_behavior`: `escalate_to_human`, `retry_once`, or `hard_fail` — determines dispatcher behavior on any error
- `audit_level`: `standard` or `elevated`; `elevated` writes additional provenance fields to the audit event

MCP and Platform Workflow: End-to-End

MCP sits at the center of every agent workflow in the platform. The table below traces how a single business task — an adjudication assist request — flows from task intake through the full MCP tool stack and back to the structured output and audit stream.

Workflow Stage	MCP Role	Servers Involved	Output
Task intake	Run spec issued with <code>tool_allowlist</code> and	None (control plane)	Approved run spec with scoped agent identity

Workflow Stage	MCP Role	Servers Involved	Output
	data_scope		
Classification	Classifier agent calls Jira to read ticket context	Jira MCP server	Ticket type, priority, and routing decision
Retrieval	Retriever agent calls Claims API and Policy corpus	Claims API, Policy corpus	Relevant claim lines and policy rules with citations
Reasoning	Reasoner agent calls Clinical corpus for code lookup	Clinical corpus, Domain RAG	Grounded reasoning chain with cited evidence
Structuring	Formatter agent calls Output write server	Output write server	Schema-conformant adjudication recommendation artifact
Audit emission	Dispatcher writes one event per tool call	Immutable audit stream	Full trace: agent, tool, input hash, output hash, eval score
Escalation (if triggered)	Escalation agent calls escalate method	Output write server, Jira MCP	HITL task created with full context attached

MCP Security Posture

Security Guarantees Enforced by the MCP Layer

- No ambient access: agents cannot call any tool outside the explicit run spec allowlist, regardless of their clearance level
- No cross-tenant tool calls: each agent identity token is scoped to a single tenant; the Dispatcher rejects any call where the agent identity does not match the server tenant context
- No standing credentials at the tool server: the Secrets Manager MCP server issues time-bound, method-scoped tokens only; the tool server never holds long-lived credentials
- PHI boundary enforcement: calls to HIPAA-tier servers require Restricted clearance in the agent identity token; Standard-clearance agents are rejected at the Dispatcher
- Write operations require explicit allowlisting: write_recommendation, write_report, and escalate are never included in the default allowlist; they must be explicitly declared in the run spec
- Every failure is audited: auth failures, allowlist rejections, schema violations, and timeouts all produce immutable audit events with reason codes

Regulated Operating Model

Aligns with: Security and Compliance Posture, HIPAA and SOC 2-class

Every design decision in the Intelligence Layer is oriented toward satisfying regulated industry requirements. Compliance is embedded in the execution model, not bolted on after the fact.

Compliance-Ready Controls

- No training on production PHI without explicit tenant policy and a de-identification pipeline
- Explainability pack per decision: inputs hash, retrieval IDs, model version, agent trace, eval scores
- Human-in-the-loop as a first-class orchestration state, not an afterthought
- Fail closed: low grounding or policy eval triggers escalation; auto-pay on high-risk paths is never permitted
- Audit replay: any decision can be reproduced from its run ID, policy version, and model registry snapshot

Alignment covers HIPAA, SOC 2-class deployments, and AI COE governance. No production PHI enters training pipelines without a governed, de-identified data lifecycle.

Reference Domain: Health Industry

Aligns with: Vertical Application, Payment Integrity and Clinical Workflows

The Health Industry is the primary reference vertical. Payment integrity, clinical review, and formulary workflows represent the highest-value, highest-precision applications of domain-aware multi-agent reasoning.

Workflow Area	Intelligence Layer Contribution
Adjudication automation	Line-level recommendations with citations to policy and claim facts; structured output for analyst accept or override
Fraud, waste, and abuse detection	Classify, retrieve similar cases, score and explain flag; precision-focused eval gates throughout
Network and contract review	Contract retrieval with rule hybrid paths; deterministic fast paths for clean-line processing
Pharmacy and specialty	Formulary and clinical corpus RAG with escalation on low-confidence decisions
Clinical document workflows	Multi-agent clinical note reasoning with strict PHI mode and mandatory human-in-loop threshold
High-throughput processing	Batch tier routing; deterministic fast path handles volume economics for clean lines

AWS Tenant Isolation: Per-Account Architecture

Each health industry tenant runs in a dedicated AWS micro-account. The diagram below shows the SCP guardrail layer, per-account security controls, observability routing, and tenant lifecycle management.

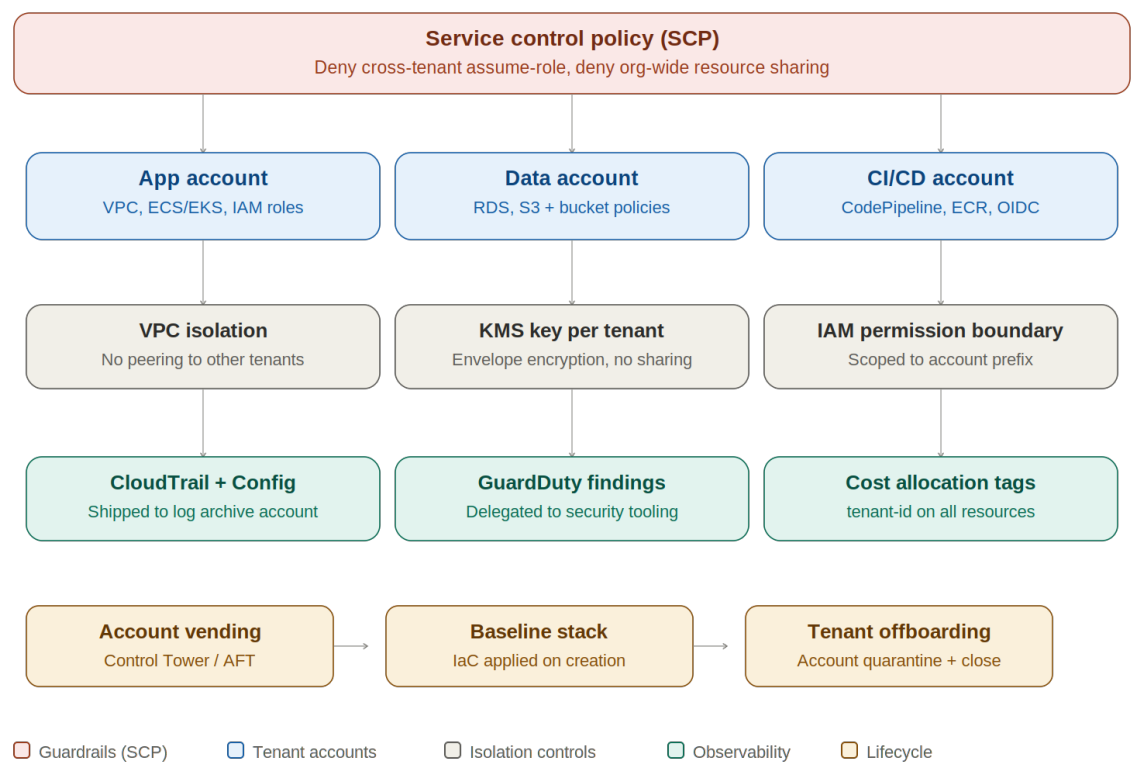


Figure 3: AWS Tenant Isolation — SCP, Account Controls, Observability, and Lifecycle

POC to Enterprise Path

POC	Establish Baseline - One pattern (e.g., FWA triage or adjudication assist) - Golden set of labeled cases established - Grounding threshold defined
Pilot	Regression Discipline - Regression CI and grounding eval live - Audit replay capability operational - Analyst human-in-loop loop instrumented and measured
Production	Full Throughput Tier - Router cost tiers active - p95 SLO enforced per claim type - Model health dashboard available for governance review
Scale	Multi-Tenant Policy Packs - Per-tenant policy packs deployed - Optional embedding fine-tune per corpus - Cross-workflow pattern reuse across client base

Secure and orchestration layers prove we can run AI safely. The Intelligence Layer proves we can trust decisions, with precision metrics, regression discipline, and explainability that satisfy payment integrity and compliance stakeholders.

Reference Domain: Recruiting Industry

Aligns with: Agent Platform Delivery, Regulated Agentic Workflows

The Recruiting Industry represents the second reference vertical: teams deploying AI agents across high-volume, document-intensive operational workflows where compliance, audit, and structured output quality are non-negotiable.

Client Need	Intelligence Layer Answer
LLM in production at scale	Registry, routing, and structured outputs with cost caps per tenant
LangGraph and MCP integrations	Agent mesh plus tool bus integration; orchestration boundary enforced
Hybrid deterministic and LLM paths	Router is the centerpiece: deterministic-first, with LLM invoked only where needed
Moving beyond demos to production	Eval plane plus recovery patterns inherited from the orchestration layer
Regulated mid-market compliance	Policy packs plus fail-closed defaults per vertical; audit replay on demand

AWS Micro-Account Strategy Overview

The diagram below shows the full AWS Organizations structure: payer and management account, shared services OU, and the per-tenant OU with App, Data, and CI/CD sub-accounts per tenant.

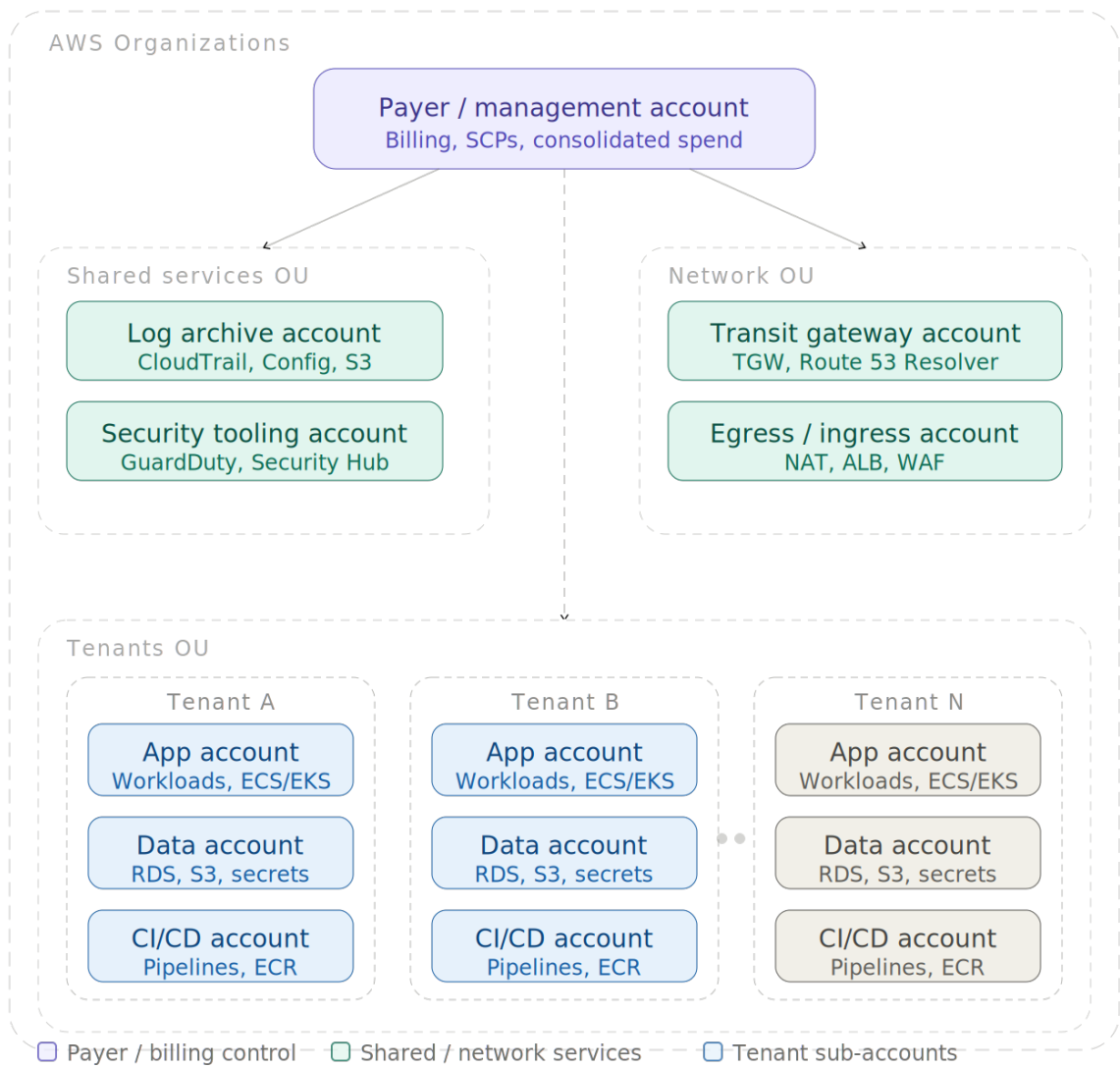


Figure 4: AWS Micro-Account Strategy — Organizations, Shared Services, and Per-Tenant Structure

The Intelligence Layer ships as modules on an existing secure and orchestration install, not a greenfield build. Faster time-to-value per client, lower integration risk, and a clear upgrade path from orchestration to reasoning.

SDLC Integration Guide

Aligns with: Developer and Engineering Guidance, Code Release and Development Lifecycle

This section maps the Intelligence Layer capabilities to each phase of the software development lifecycle. It is designed for developers, engineers, and platform leads who need to understand where and how the Intelligence Layer plugs into their existing release process.

SDLC Design Principles

- Every phase produces a structured artifact: no phase exits without a schema-conformant output and an eval score
- Every gate is policy-driven: promotion criteria are encoded in versioned policy packs, not ad hoc decisions
- Every action is auditable: agent trace, citations, model version, and eval scores are written per phase
- Fail closed at every gate: low confidence or failed eval routes to human review, never silently passes
- Retrospective feeds forward: post-release learnings update the golden set for the next cycle

Phase-by-Phase Contribution

SDLC Phase	Intelligence Layer Contribution
Planning and Requirements	• Defines agent workflow templates for each SDLC phase • Establishes golden sets aligned to each release type (feature, hotfix, major) • Encodes acceptance criteria for each phase gate into policy packs
Development	• Classifies incoming Jira tickets and routes to the correct agent workflow • Retrieves relevant code standards, security policies, and past decisions from the RAG corpus • Flags drift from coding standards before code review begins
Code Review and Static Analysis	• Runs structured output checks against review criteria schema • Cross-references PR content against policy corpus and regression test baselines • Produces an explainability pack per review: what was checked, what was flagged, citations
CI/CD and Build Pipeline	• Monitors pipeline events from the CI/CD account via SQS and the orchestration engine • Routes build failures to the correct triage agent based on failure signature • Enforces policy gates before any artifact is promoted to staging or production
Testing and QA	• Applies golden-set regression eval against test results on every build • Detects regression against prior release baselines using the eval plane • Escalates to human review when grounding or coverage scores fall below threshold

SDLC Phase	Intelligence Layer Contribution
Deployment and Release	- Validates release artifacts against the output schema before promotion - Writes an audit event per deployment: model version, policy version, eval scores, approver - Supports rollback decision by replaying the prior release audit record
Monitoring and Incident Response	- Routes incident signals from Slack, PagerDuty, or SQS to the triage agent mesh - Enriches incidents with relevant historical context from the RAG corpus - Generates structured incident reports with citations and recommended next steps
Post-Release and Retrospective	- Synthesizes release data across all phases into an audit-ready retrospective artifact - Updates the golden set baseline with learnings from the release cycle - Feeds quality signals back to the model health dashboard for continuous improvement

Developer Integration Points

The following integration points are the primary touchdowns for engineering teams adopting the Intelligence Layer within their existing toolchain.

Integration Point	How to Connect	What It Enables
Jira webhook or SQS event	Publish Jira ticket events to the Orchestrator intake SQS queue via a lightweight Lambda or webhook adapter	Automatic ticket classification, routing, and enrichment without changing developer workflow
GitHub Actions or CodePipeline	Add an Intelligence Layer policy gate step to the pipeline YAML; the step calls the run spec API and blocks on eval score	Automated policy compliance checks, regression gates, and artifact validation before promotion
Slack or Teams notification	Subscribe the Orchestrator to incident channels; route alerts to the triage agent mesh	Structured incident enrichment, historical context retrieval, and recommended actions on every alert
CloudWatch and GuardDuty	Route findings to the tenant SQS queue; the Orchestrator picks up and routes to the security triage agent	Automated finding enrichment, severity scoring, and escalation with full audit trail
Pull request review	Integrate via GitHub App or webhook; the review agent retrieves policy corpus and produces a structured review artifact	Policy-grounded code review with mandatory citations; reviewer sees what was checked and why
Release gate API	Call the release gate endpoint from the pipeline before any artifact is promoted; gate returns pass, escalate, or fail	Deterministic, auditable release decisions with a full explainability pack per promotion event

Credential and Identity Model for Engineering Teams

How Engineers Authenticate to the Intelligence Layer

- Human developers authenticate via SSO or OIDC; the Control Plane validates identity before issuing scoped run specs
- CI/CD pipelines authenticate via OIDC federation from the CI/CD account; no standing credentials are stored in the pipeline
- Each agent run receives short-lived, scoped credentials via the Orchestrator; credentials expire at run completion
- All credential issuance and revocation events are written to the immutable audit log in the Log Archive account
- Break-glass access requires dual approval and triggers an immediate audit event with elevated retention

Golden Set Management for Engineering Teams

The golden set is the foundation of the eval plane. Engineering teams own and maintain the golden set for their vertical or workflow. The following process governs golden set lifecycle.

Step	Owner	Tooling
Define acceptance criteria per release type	Product and engineering lead	Policy pack editor; criteria encoded as versioned JSON schema
Seed initial golden set from labeled historical cases	Data and ML team	Ingestion pipeline; cases stored in the tenant RAG corpus
Run baseline eval on each build	CI/CD pipeline (automated)	Eval plane API; results written to the audit log
Review and triage failed evals	Engineer or QA lead	Model health dashboard; escalation routed via Slack or Jira
Update golden set after each release	Data and ML team	Post-release synthesis agent; learnings merged into next baseline
Promote updated golden set to production	Engineering lead approval	Policy pack versioning; promotion gated by dual approval

Differentiation

Aligns with: Competitive Positioning, Operational Trust over Agent Capability

The Intelligence Layer does not compete on raw model capability. It competes on measurable, auditable, explainable decision quality, the bar that regulated enterprises cannot waive.

Alternative Approach	Weakness	Intelligence Layer
Raw LLM API and prompts	No regression, no grounding discipline, no schema enforcement	Schema, eval, and mandatory citations on all material outputs
RPA-only automation	No unstructured reasoning; breaks on document variance	Hybrid router adds LLM exactly where needed
Generic agent frameworks	Chaos at multi-agent scale; no message flood control	Mesh plus orchestrator boundary; no unbounded agent chatter
Copilot wrappers	Weak audit replay; no per-decision evidence trail	Evidence events per agent step; full audit replay available
Bolt-on RAG	Retrieval without domain eval; no golden-set validation	Tuned retrieval plus golden sets; grounding score gated on promotion

Structural Moat

- Secure, Orchestration, and Intelligence: three layers that compound; each deployment adds golden sets, calibrated retrievers, and operator feedback loops
- Domain eval libraries and policy packs improve with every regulated installation
- Cross-tenant pattern reuse (not raw data) creates durable differentiation across the install base

Roadmap and Milestones

Aligns with: Roadmap, Three Phases to Scale

Phase 2a 0 to 4 Months	Intelligence MVP - Hybrid router v1 using template plus confidence rules - Three agent patterns: Retrieve, Reason, and Structure; Classify and Branch; Extract and Validate - Primary model production adapter with structured output and cost caps - Eval MVP: golden-set regression plus grounding score - One vertical policy pack for health industry or platform operations
Phase 2b 4 to 8 Months	Production Hardening - MCP tool catalog for claims and operational APIs - Model health dashboard for governance and product review - Batch throughput tier plus cost-per-decision metrics - Embedding fine-tune pipeline per tenant (optional add-on) - Second vertical policy pack
Phase 2c 8 to 14 Months	Scale - LoRA and SFT for narrow classifiers per vertical - Cross-tenant pattern marketplace covering policy packs, not raw data - Long-run workflow memory patterns coordinated by orchestration - External partner API for intelligence-only integrations

Success Metrics

Aligns with: Key Performance Indicators, Quality, Operations, and Governance

Category	KPI Examples
Quality	Grounding score at or above defined threshold; precision on domain flags; analyst override rate trending down
Operations	p95 latency by SLA tier; cost per claim line or ticket; batch throughput; escalation rate by work type
Governance	100% audit replay success; zero unlogged auto-actions on high-risk paths; HITL rate on low-confidence paths
Product	POC to production conversion time; regression catch rate pre-deploy; time to first golden set established
Commercial	Reusable policy packs shipped; reduction in per-client custom prompt work; ACV expansion from intelligence tier
Model Health	Drift score per tenant and model; version mismatch alerts; cost and latency trend by model tier

Team and Build Assumptions

Aligns with: Team, Roles and Build Stance

Role	Focus
Agentic AI Lead	Router philosophy, agent mesh standards, eval governance, executive safety narrative and COE alignment
Agentic AI Engineers	Python services, model and MCP integrations, orchestration debugging, token economics at throughput scale
Data and ML	Pipelines for unstructured medical and claims ingest; embedding fine-tune; golden set construction and maintenance
Product (VP-class)	Roadmap across health industry workflows; COE alignment; ROI narratives for executive and compliance stakeholders

Build stance: Extend the existing orchestration layer, not a fork. The Intelligence Layer is a service domain with clear APIs into Layers 1 and 2. No parallel infrastructure build required.

Strategic Options

Aligns with: Go-To-Market, How This Asset Is Deployed

Option	Description
A: Internal Platform	Productize for multi-tenant deployments. Full control over roadmap, eval libraries, and policy pack quality accumulation across verticals.
B: Partner Embed	License intelligence APIs to SI and agent platforms missing regulated reasoning. Fast distribution with defined commercial boundaries.
C: Domain JV	Co-develop vertical-specific policy packs with a health industry or regulated services partner. Shared golden sets, shared roadmap, joint compliance authority.
D: Platform Narrative	Layers 1 and 2 are built. Layer 3 is the roadmap: a clear, differentiated platform story for enterprise agentic AI across regulated verticals.

Diligence Appendix

The following materials are available upon request for technical diligence and partner conversations:

Available on Request

- Trust boundary diagram: secure, orchestration, and intelligence data flows
- Sample health industry workflow with adjudication and FWA eval gates
- Golden-set methodology and regression CI outline
- Model registry and governance checklist
- Data handling matrix for training versus inference in regulated environments
- Pilot blueprint with entry criteria, success metrics, and conversion triggers
- SDLC integration runbooks for Jira, GitHub Actions, and CodePipeline
- Engineering onboarding guide for golden set seeding and eval configuration